

Softwarekonstruktion WS 16/17 – Übung 6

Ausgabe: 17.01.2017, Abgabe: 30.01.2017 12:00 Uhr

Präsenzübung

6.1 Signatur und Algebra

Gegeben Sei folgende Signatur:

```
 $\Sigma =$  sorts  int  
      opns  zero:  $\rightarrow$  int  
            succ: int  $\rightarrow$  int  
            inv: int  $\rightarrow$  int
```

Geben Sie eine Σ -Algebra A an, so dass **zero**, **succ** und **inv** die generierenden Operationen der Menge der ganzen Zahlen sind.

Hinweis: Die Operation **inv** soll einen Vorzeichenwechsel für alle Zahlen außer 0 bewirken.

Lösungsvorschlag

$$\begin{aligned} A_{\text{int}} &:= \mathbb{Z} \\ \text{zero}_A &:= 0 \\ \text{succ}_A(z) &:= 1 + z, \forall z \in \mathbb{Z} \\ \text{inv}_A(z) &:= -z, \forall z \in \mathbb{Z} \setminus 0 \end{aligned}$$

6.2 Algebraische Spezifikation: Stack

Geben Sie eine algebraische Spezifikation für eine Stack-Datenstruktur mit den folgenden Operationen an:

1. **empty** : erzeugt einen leeren Stack
2. **push** : legt ein Element auf den Stack
3. **top** : gibt das oberste Element des Stacks zurück, sofern es eines gibt, sonst 0
4. **pop** : entfernt das oberste Element des Stacks, sofern es eines gibt
5. **size** : liefert die Anzahl der Elemente im Stack

Lösungsvorschlag

```
 $SPEC_{\text{stack}} =$  sorts  stack, elem, nat  
      opns  empty:  $\rightarrow$  stack
```

```
push: stack  $\times$  elem  $\rightarrow$  stack
top: stack  $\rightarrow$  elem  $\cup$  nat
pop: stack  $\rightarrow$  stack
size: stack  $\rightarrow$  nat
zero:  $\rightarrow$  nat
succ: nat  $\rightarrow$  nat
vars  s: stack
       e: elem
eqns size(empty()) = 0
       size(push(s,e)) = succ(size(s))
       top(push(s,e)) = e
       top(empty()) = 0
       pop(push(s,e)) = s
       pop(empty()) = empty()
```

Hausübung

6.3 Signatur und Algebra: Rationale Zahlen (2 Punkte)

Gegeben sei folgende Signatur der rationalen Zahlen:

```
Σ = sorts rat
    opns zero: → rat
         one: → rat
         add: rat × rat → rat
         sub: rat × rat → rat
         mult: rat × rat → rat
         div: rat × rat → rat
```

Geben Sie eine Algebra an, die die Signatur interpretiert.

Lösungsvorschlag

$$\begin{aligned}
 A_{\text{rat}} &:= \mathbb{Q} \\
 \text{zero}_A &:= 0 \\
 \text{one}_A &:= 1 \\
 \text{add}_A(x, y) &:= x + y & \forall x, y \in \mathbb{Q} \\
 \text{sub}_A(x, y) &:= x - y & \forall x, y \in \mathbb{Q} \\
 \text{mult}_A(x, y) &:= x \cdot y & \forall x, y \in \mathbb{Q} \\
 \text{div}_A(x, y) &:= \frac{x}{y} & \forall x, y \in \mathbb{Q}, y \neq 0
 \end{aligned}$$

Dies ist nur eine von vielen interpretierenden Algebren. Prinzipiell ginge als Minimalbeispiel z.B. auch:

$$\begin{aligned}
 B_{\text{rat}} &:= \{\phi\} \\
 \text{zero}_B &:= \phi \\
 \text{one}_B &:= \phi \\
 \text{add}_B(x, y) &:= \phi \quad \forall x, y \in \{\phi\} \\
 \text{sub}_B(x, y) &:= \phi \quad \forall x, y \in \{\phi\} \\
 \text{mult}_B(x, y) &:= \phi \quad \forall x, y \in \{\phi\} \\
 \text{div}_B(x, y) &:= \phi \quad \forall x, y \in \{\phi\}
 \end{aligned}$$

6.4 Algebraische Spezifikation: FIFO (6 Punkte)

1. Geben Sie für eine FIFO-Warteschlange eine vollständige algebraische Spezifikation mit folgenden Operationen an:

- **create** (zum Erzeugen einer Warteschlange)
- **enqueue** (zum Hinzufügen eines Objekts)
- **dequeue** (zum Entfernen eines Objektes)
- **front** (Zurückholen eines Objektes)
- **isempty** (zur Abfrage, ob die Warteschlange leer ist).

Lösungsvorschlag

- Algebra FIFO
 - Introduces sorts **bool**, **elem**, **queue**
- Operations
 - **create**: $\rightarrow \mathbf{queue}$
 - **enqueue**: $\mathbf{queue} \times \mathbf{elem} \rightarrow \mathbf{queue}$
 - **dequeue**: $\mathbf{queue} \rightarrow \mathbf{queue}$
 - **front**: $\mathbf{queue} \rightarrow \mathbf{elem}$
 - **isempty**: $\mathbf{queue} \rightarrow \mathbf{bool}$
- Constraints **create**, **enqueue**, **dequeue**, **front**, **isempty**
so that $\forall q \in \mathbf{queue}; \forall e_1, e_2 \in \mathbf{elem}$
 - **front**(**enqueue**(**create**(), e_1)) = e_1
 - **front**(**create**()) = undefined
 - **front**(**enqueue**(**enqueue**(q, e_1), e_2)) = **front**(**enqueue**(q, e_1))
 - **dequeue**(**create**()) = **create**() //oder undefined
 - **dequeue**(**enqueue**(**create**(), e_1)) = **create**()
 - **dequeue**(**enqueue**(**enqueue**(q, e_1), e_2)) = **enqueue**(**dequeue**(**enqueue**(q, e_1)), e_2)
 - **isempty**(**create**()) = true
 - **isempty**(**enqueue**(q, e_1)) = false

6.5 Algebraische Spezifikation: Eigenschaften von Algebren (4 Punkte)

Vervollständigen Sie:

1. Eine algebraische Spezifikation ist *redundant*, wenn ...
2. Eine algebraische Spezifikation ist *widersprüchlich*, wenn ...
3. Eine Algebra folgt bzgl. einer algebraischen Spezifikation dem Prinzip *no-junk*, wenn ...
4. Eine Algebra folgt bzgl. einer algebraischen Spezifikation dem Prinzip *no-confusion*, wenn ...

Lösungsvorschlag

1. Eine algebraische Spezifikation ist *redundant*, wenn sich ein Axiom entfernen lässt und die Menge der beweisbaren Aussagen gleich bleibt.
2. Eine algebraische Spezifikation ist *widersprüchlich*, wenn sich beweisen lässt, dass zwei als unterschiedlich definierte Elemente gleich sind.
3. Eine Algebra folgt bzgl. einer algebraischen Spezifikation dem Prinzip *no-junk*, wenn sie nur Elemente enthält, die durch Terme der Signatur bezeichnet werden können.
4. Eine Algebra folgt bzgl. einer algebraischen Spezifikation dem Prinzip *no-confusion*, wenn nur diejenigen Eigenschaften gelten, die durch die Spezifikation ausgedrückt werden.

6.6 Algebraische Spezifikation: Auswertung I (5 Punkte)

Gegeben sei die Spezifikation eines Stacks, die insbesondere folgende Axiome enthält. Zusätzlich zu den normalen Funktionen eines Stacks, soll es möglich sein, mit `bottom` den Stack leerräumen und das unterste Element auszugeben:

$$\text{I } \text{pop}(\text{push}(\text{st}, e)) = \text{st}$$

$$\text{II } \text{bottom}(\text{push}(\text{create}(), e)) = e$$

$$\text{III } \text{bottom}(\text{st}) = \text{bottom}(\text{pop}(\text{st}))$$

Zeigen Sie, wie sich mit diesen Axiomen der Funktionswert `bottom([1,2,3,4])` berechnen lässt.

$$\begin{aligned} \text{bottom}([1,2,3,4]) &= \text{bottom}(\text{pop}([1,2,3,4])) \text{ (III)} \\ &= \text{bottom}([1,2,3]) \text{ (I)} \\ &= \text{bottom}(\text{pop}([1,2,3])) \text{ (III)} \\ &= \text{bottom}([1,2]) \text{ (I)} \\ &= \text{bottom}(\text{pop}([1,2])) \text{ (III)} \\ &= \text{bottom}([1]) \text{ (I)} \\ &= \text{bottom}(\text{push}(\text{create}(), 1)) \text{ (II)} \\ &= 1 \end{aligned}$$

6.7 Algebraische Spezifikation: Auswertung II (5 Punkte)

Gegeben sei die Spezifikation einer Liste, die insbesondere folgende Axiome enthält.

$$\text{I } \text{con}(\text{L}, \text{new}()) = \text{L}$$

$$\text{II } \text{con}(\text{L}_1, \text{add}(\text{L}_2, e)) = \text{add}(\text{con}(\text{L}_1, \text{L}_2), e)$$

Zeigen Sie, dass sich die Gleichung (III) $\text{con}(\text{new}(), \text{L}) = \text{L}$ aus den Axiomen ableiten lässt.

Es wird davon ausgegangen, dass `new()` eine neue, leere Liste erzeugt und dass `add(L, e)` einer beliebigen Liste L ein Element e hinzufügt und in jedem Fall eine nichtleere Liste zurückgibt.

- Für *leere Listen* L gilt $L = \text{new}()$ und damit

$$\begin{aligned} \text{con}(\text{new}(), L) &= \text{con}(\text{new}(), \text{new}()) && \text{wegen (I) folgt} \\ &= \text{new}() = L \end{aligned}$$

- Für *nichtleere Listen* L wird der Beweis durch strukturelle Induktion erbracht:

- Induktionsanfang: Für Listen L der Länge 1 gilt $L = \text{add}(\text{new}(), e)$ und damit

$$\begin{aligned} \text{con}(\text{new}(), L) &= \text{con}(\text{new}(), \text{add}(\text{new}(), e)) && \text{wegen (II) folgt} \\ &= \text{add}(\text{con}(\text{new}(), \text{new}()), e) && \text{wegen (I) folgt} \\ &= \text{add}(\text{new}(), e) = L \end{aligned}$$

- Induktionsvoraussetzung: Gleichung (III) gelte für Listen L beliebiger Länge n .
- Induktionsschritt: Für Listen L' mit Länge $n+1$ gilt, dass sie sich als $L' = \text{add}(L, e)$ darstellen lassen und somit:

$$\begin{aligned} \text{con}(\text{new}(), L') &= \text{con}(\text{new}(), \text{add}(L, e)) && \text{wegen (II) folgt} \\ &= \text{add}(\text{con}(\text{new}(), L), e) \\ &\stackrel{IV}{=} \text{add}(L, e) = L' \end{aligned}$$

Gleichung (III) gilt also für Listen beliebiger Länge.

6.8 Entwurfsprinzipien: Beispiele (8 Punkte)

Beschreiben Sie für drei der folgenden Entwurfsprinzipien jeweils ein Beispiel:

1. Separation of Concerns
2. Single Responsibility Principle
3. Open-Closed Principle
4. Interface-Segregation Principle
5. Dependency Inversion Principle
6. Law of Demeter
7. Design by Contract
8. Don't Repeat Yourself
9. Self-documentation
10. Design for change